Retour sur la technologie utilisée

Notre retour après 36 heures de construction. Chaque point vient d'un fait relevé pendant le hackathon, avec ce qu'il nous a coûté et la façon dont nous l'avons traité. Rien ici n'est une impression générale.

Pile concernée : un L1 Avalanche privé (Subnet-EVM, avalanchego v1.15.1, enregistré sur la P-Chain de Fuji, un validateur sur AWS EC2), Foundry pour les contrats, Amazon Bedrock pour les vérificateurs LLM.

Avalanche : ce qui nous a servi

1. **La liste d'émetteurs admis (TxAllowList) est notre meilleure preuve.** Une ligne de genèse, et le nœud refuse tout émetteur non admis avant d'atteindre un contrat. Une adresse financée en gaz mais non admise reçoit `cannot issue transaction from non-allow listed address`. Le message est clair et se montre tel quel à un jury.
2. **Elle change aussi le modèle de menace des contrats.** Deux fonctions de notre coffre n'exigent aucun rôle, `expire` et `fund`. Sur le L1, elles ne sont ouvertes qu'aux adresses admises. Une surface publique sur une chaîne ouverte devient une surface restreinte, sans une ligne de Solidity.
3. **Le Native Minter évite un jeton ERC-20.** Le guichet et les récompenses frappent le jeton natif depuis un contrat. Frais, cautions et récompenses sont dans la même unité que le gaz.
4. **Un seul validateur suffit pour démarrer.** Le L1 tourne avec son validateur de conversion, pour un coût continu de l'ordre de 1,33 AVAX par mois sur Fuji. Il se vérifie publiquement sur l'API de la P-Chain.
5. **`validatorOnly` tient ce qu'il promet.** Un nœud non validateur, pair direct du nôtre, n'a jamais reçu la chaîne. La contre-épreuve avec `validatorOnly: false` synchronise.
6. **Le nom du jeton natif n'est pas dans la genèse.** Nous avons renommé le jeton en cours de route sans rien redéployer. Une bonne surprise.
7. **La compatibilité EVM est réelle.** Les mêmes scripts Foundry déploient en local et sur le L1, en changeant le RPC et la clé.

Avalanche : ce qui nous a coûté du temps

1. **La CLI ne suivait pas Fuji.** La CLI Avalanche v1.9.6 embarque avalanchego 1.14.0 et Subnet-EVM v0.8.0 (rpcchainvm 44). Fuji tournait en avalanchego 1.15.1 (rpcchainvm 46). Le L1 sur Fuji ne s'est donc pas fait avec la CLI. Nous sommes passés par Platform CLI et les binaires de la release officielle. La CLI est restée utile pour le réseau local seulement, sur une version différente de celle de la démo.
2. **Un L1 sans trafic ne produit aucun bloc.** `block.timestamp` reste figé. Nos fenêtres d'engagement, de révélation et de contestation ne s'écoulaient plus, et toute simulation échouait en `WrongPhase` ou `TooEarly`. Nous avons dû écrire un battement de blocs : une transaction neutre toutes les cinq secondes, depuis une adresse admise dédiée. C'est le point qui nous a le plus surpris, et un contrat à échéances ne peut pas l'ignorer.
3. **anvil masquait ce défaut.** Sur anvil, `eth_estimateGas` s'exécute à l'heure courante, pas sur le dernier bloc. Un envoi sans gaz fixe y passait et échouait sur le L1. Nos tests locaux étaient verts sur un code faux. Nous avons fixé le gaz des appels sensibles au temps et ajouté des tests sur chaîne inactive.
4. **Un contrat qui frappe n'est pas portable.** Le contrat doit recevoir le rôle sur le précompilé, sinon la frappe échoue. Et la C-Chain n'a pas ce précompilé : notre guichet et nos récompenses n'y fonctionnent pas. Nous avons renoncé à une démo sur la C-Chain de Fuji pour cette raison.

5. **« Sans ICM » ne veut pas dire « sans Warp ».** Nous voulions une chaîne sans messagerie inter-chaînes. Le précompilé Warp doit pourtant rester actif dans la genèse : le ValidatorManager l'appelle pour parler à la P-Chain. La frontière entre Warp, ICM et Teleporter nous a demandé une lecture attentive de la documentation.
6. **Le jeu de validateurs est figé tant que le ValidatorManager n'est pas initialisé.** Le L1 tourne sans cela, ce qui est commode. Mais ajouter un validateur demande cette initialisation, puis une machine par validateur. Nous ne l'avons pas fait : notre démo n'a aucune tolérance à la panne.
7. **Le réseau du site bloquait le pair-à-pair.** Le Wi-Fi invité filtrait le port 9651 en sortie : le nœud lancé sur le portable n'a trouvé aucun pair. Nous avons déplacé le validateur sur EC2 en copiant ses clés de staking, sans nouvelle transaction sur la P-Chain. L'amorçage de la P-Chain a pris dix minutes sur une instance à 8 Go.
8. **La clé de test `ewoq` ne s'exporte pas.** `avalanche key export ewoq` échoue, la clé étant incluse dans le binaire. Comme elle est publique, la genèse du L1 de démo a reçu un administrateur distinct de celle du local.
9. **La limite de 24 576 octets par contrat.** Notre marché à tirage au sort la dépassait sans optimiseur. Elle impose `--optimize --optimizer-runs 200`.
10. **Pas d'aléa natif utilisé.** Le comité est tiré du hash d'un de nos blocs. Avec un seul validateur, celui-ci peut influer sur le tirage. Nous le disons dans nos limites, sans l'avoir corrigé.

Foundry

- **Ni les invariants ni Slither n'ont vu notre défaut le plus sérieux.** Le coffre pouvait verser, à l'approbation, un placement devenu incompatible avec la réserve. C'est la relecture humaine qui l'a trouvé. Notre campagne d'invariants ne l'atteignait pas : elle n'inscrivait aucune facture et ne relevait jamais la réserve en cours de séquence. Slither ne voit pas une erreur de logique métier. Nous avons écrit deux tests rouges, corrigé, puis étendu la campagne. Un invariant ne vaut que par les actions qu'on lui donne.
- **Slither demande un tri.** 36 et 57 alertes sur nos deux ensembles de contrats. Il a fallu qualifier chacune : faux positif, limite acceptée ou constat.
- `forge script` déploie sur le L1 comme en local. La seule différence de comportement vient d'anvil, décrite plus haut.

Amazon Bedrock

Banc de dix pièces par modèle, puis injections jouées sur l'évaluateur.

- **L'API Converse rend les modèles interchangeables.** Un vérificateur change de modèle par une variable. C'est ce qui a permis un comité de trois modèles et de deux fournisseurs.
- **Un modèle rapide et bien formaté peut mal juger.** Amazon Nova Lite passait nos critères de latence et de format. Il acceptait les dix pièces, attaques comprises, et cédait à deux injections sur trois. Nous l'avons retiré.
- **Le format n'est pas acquis.** Amazon Nova Pro a rendu un JSON invalide quatre fois sur dix. Retiré aussi.
- **Les voix ne se valent pas.** Mistral Pixtral Large vote « accepté » sans pièce jointe, là où les deux modèles d'Anthropic demandent une validation humaine. C'est la voix la plus indulgente du comité.
- **La latence tient.** Claude Haiku 4.5 : 2,3 s au 95e centile. Claude Sonnet 4.5 : 4,3 s. Pour un seuil de 10 s.
- **Les identifiants temporaires expirent.** Ils doivent être rafraîchis avant chaque démo. Un évaluateur sans identifiants valides vote « validation humaine », ce qui est le bon repli, mais affaiblit la

démonstration.

Ce que nous aurions aimé trouver

1. Une CLI alignée sur la version de Fuji, ou un avertissement dès la création du réseau.
2. Une option de production de blocs périodiques pour un L1 de test, ou une page de documentation qui nomme le problème des échéances sur une chaîne inactive.
3. Un nœud local de développement qui porte les précompilés et le comportement temporel d'un L1, pour que les tests locaux échouent là où le L1 échoue.
4. Une liste de contrôle « contrat qui appelle un précompilé » : rôles à accorder, et ce qui cesse de marcher hors d'un L1.